

Web API Hands-On – Answers

Module 4: CRUD Operations – PUT (Update) with Validation

Step 1: Model Classes (same as previous task)

Models/Employee.cs

```
public class Employee { public int Id {  
    get; set; } public string Name { get; set;  
    } public int Salary { get; set; } public  
    bool Permanent { get; set; } public  
    Department Department { get; set; } public  
    List Skills { get; set; } public DateTime  
    DateOfBirth { get; set; } }
```

Step 2: Hardcoded Employee List (static field in controller)

Controllers/EmployeeController.cs – static list

```
private static List _employees = new List { new Employee  
    { Id = 1, Name = "Alice", Salary = 75000, Permanent =  
    true,  
    DateOfBirth = new DateTime(1990, 5, 10), Department = new  
    Department { DeptId = 1, DeptName = "IT" }, Skills = new  
    List { new Skill { SkillId = 1, SkillName = "C#" }, new  
    Skill { SkillId = 2, SkillName = "ASP.NET" } } }, new  
    Employee { Id = 2, Name = "Bob", Salary = 60000, Permanent  
    = false,  
    DateOfBirth = new DateTime(1995, 3, 22), Department = new  
    Department { DeptId = 2, DeptName = "HR" }, Skills = new  
    List { new Skill { SkillId = 3, SkillName = "Excel" } } },  
    new Employee {  
    Id = 3, Name = "Charlie", Salary = 90000, Permanent = true,  
    DateOfBirth = new DateTime(1988, 11, 5), Department = new  
    Department { DeptId = 1, DeptName = "IT" }, Skills = new  
    List { new Skill { SkillId = 4, SkillName = "Azure" } }  
    }  
};
```

Step 3: GET Action Method (recap)

The GET method returns the full list. This is unchanged from the previous task.

Controllers/EmployeeController.cs – GET

```
[HttpGet]
[ProducesResponseType(typeof(List), 200)]
public ActionResult< > Get() { return
Ok(_employees); }
```

Step 4: PUT Action Method – Update Employee

- Accepts id from the route and Employee object from the request body ([FromBody]).
- Returns ActionResult so both the updated object and error results can be returned.
- Validates that id <= 0 → BadRequest with 'Invalid employee id'.
- Validates that id does not exist in list → BadRequest with same message.
- If id is valid, updates the matching record in the list and returns it.

Controllers/EmployeeController.cs – PUT

```
// PUT api/employee/1
[HttpPut("{id}")]
[ProducesResponseType(typeof(Employee), 200)]
[ProducesResponseType(400)] public ActionResult Put(int
id, [FromBody] Employee emp) { // Validation 1: id must
be greater than 0 if (id <= 0) { return
BadRequest("Invalid employee id"); }

// Validation 2: id must exist in the list var
existing = _employees.Find(e => e.Id == id); if
(existing == null) { return BadRequest("Invalid
employee id"); }

// Update the fields from the request body

existing.Name = emp.Name;
existing.Salary = emp.Salary;
existing.Permanent = emp.Permanent;
existing.Department = emp.Department;
existing.Skills = emp.Skills;
existing.DateOfBirth = emp.DateOfBirth;

// Return the updated employee
return Ok(existing);
}
```

Step 5: Full EmployeeController (GET + PUT)

Controllers/EmployeeController.cs – complete file

```

using System; using
System.Collections.Generic;
using
Microsoft.AspNetCore.Mvc;
using WebApiDemo.Models;

namespace WebApiDemo.Controllers {
[Route("api/[controller]")] [ApiController] public class
EmployeeController : ControllerBase { private static List
_employees = new List { new Employee { Id=1, Name="Alice",
Salary=75000, Permanent=true, DateOfBirth=new
DateTime(1990,5,10),
Department=new Department{DeptId=1,DeptName="IT"},
Skills=new List{new Skill{SkillId=1,SkillName="C#"}}, new
Employee { Id=2, Name="Bob", Salary=60000, Permanent=false,
DateOfBirth=new DateTime(1995,3,22),
Department=new Department{DeptId=2,DeptName="HR"}, Skills=new
List{new Skill{SkillId=3,SkillName="Excel"}}, new Employee {
Id=3, Name="Charlie", Salary=90000, Permanent=true,
DateOfBirth=new DateTime(1988,11,5),
Department=new Department{DeptId=1,DeptName="IT"},
Skills=new List{new Skill{SkillId=4,SkillName="Azure"}}
}};

// GET api/employee

[HttpGet]
[ProducesResponseType(typeof(List),
200)] public ActionResult> Get() {
return Ok(_employees);
}
}

```

```

}

// PUT api/employee/{id}
[HttpPut("{id}")]
[ProducesResponseType(typeof(Employee), 200)]
[ProducesResponseType(400)] public ActionResult
Put(int id, [FromBody] Employee emp) { if (id <= 0)
return BadRequest("Invalid employee id");

var existing = _employees.Find(e => e.Id ==
id); if (existing == null) return
BadRequest("Invalid employee id");

existing.Name = emp.Name;
existing.Salary = emp.Salary;
existing.Permanent = emp.Permanent;
existing.Department = emp.Department;
existing.Skills = emp.Skills;
existing.DateOfBirth =
emp.DateOfBirth;

return
Ok(existing); }
}
}

```

Step 6: Testing with Swagger and Postman

Test Case 1 – id <= 0 (Bad Request) Swagger:

PUT /api/employee/0 or PUT /api/employee/-1

Request body sent:

Request JSON body

```

{
  "name": "Test",
  "salary": 50000,
  "permanent": true, "department": { "deptId": 1,
  "deptName": "IT" },
  "skills": [], "dateOfBirth": "1990-
01-01T00:00:00"
}

```

[Swagger / Postman – Case 1: id = 0]

```

PUT http://localhost:5001/api/employee/0
Response: 400 Bad Request
"Invalid employee id"

```

Test Case 2 – id > 0 but does not exist in list (Bad Request)

Swagger: PUT /api/employee/99 (id 99 is not in the hardcoded list)

[Swagger / Postman – Case 2: id not found]

```
PUT http://localhost:5001/api/employee/99

Response: 400 Bad Request

"Invalid employee id"
```

Test Case 3 – Valid id, successful update (200 OK)

Swagger: PUT /api/employee/1 with updated data in request body:

Request JSON body – updating Alice (id=1)

```
{
  "id": 1,
  "name": "Alice Updated",
  "salary": 85000,
  "permanent": true, "department": { "deptId": 1,
  "deptName": "IT" },
  "skills": [
    { "skillId": 1, "skillName": "C#" }, {
    "skillId": 5, "skillName": "Docker" }
  ], "dateOfBirth": "1990-05-
  10T00:00:00"
}
```

[Swagger / Postman – Case 3: Successful Update]

```
PUT http://localhost:5001/api/employee/1

Response: 200 OK

{
  "id": 1,
```

```
  "name": "Alice Updated",
  "salary": 85000,
  "permanent": true, "department": { "deptId": 1,
  "deptName": "IT" },
  "skills": [
    { "skillId": 1, "skillName": "C#" }, {
    "skillId": 5, "skillName": "Docker" }
  ], "dateOfBirth": "1990-05-
  10T00:00:00"
}
```

Verify update with GET

After the PUT call above, call GET /api/employee to confirm the change is reflected:

[GET /api/employee after PUT – confirms update persisted]

```
GET http://localhost:5001/api/employee

Response: 200 OK

[
  { "id": 1, "name": "Alice Updated", "salary": 85000,
    ...
  },
  { "id": 2, "name": "Bob", "salary": 60000,
    ...
  },
  { "id": 3, "name": "Charlie", "salary": 90000,
    ...
  }
]
```

Swagger UI – PUT method description

In Swagger UI the PUT /api/employee/{id} section shows:

[Swagger UI – PUT /api/employee/{id} documentation]

```
PUT /api/employee/{id}

Parameters: id (path,
required, integer)

Request body (application/json): Employee schema { id, name, salary, permanent,
department, skills, dateOfBirth }

Responses: 200 - Returns updated
Employee object
400 - Invalid employee id
```